



CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir



Habib University, Fall 2025

Warmup

What will be the output of this program? Any problems with **main()**?

Is **Derived::getName()** overriding **Base::getName()**?

```
class Base
{
public:
    virtual std::string getName() const { return "Base"; }

    virtual ~Base() = default;
};
```

```
class Derived : public Base
{
public:
    std::string getName() { return "Derived"; }

    // Optional to write.
    virtual ~Derived() = default;
};
```

If I don't have any non-trivial implementation for the dtor, why bother to write it out like this?

```
int main()
{
    Base* p { new Derived };

    std::cout << p->getName();

    delete p;
    p = nullptr;

    return 0;
}
```

“default” tells the compiler to provide a destructor. “default” can be used in other situations as well.

I could have alternatively written the following.

```
virtual ~Base() {}
virtual ~Derived() {}
```

1 2023\CS224
Base
PS C:\Heore\

What will be the output of this program? Any problems with **main()**?

Is **Derived::getName()** overriding **Base::getName()**?

1 2023/03/24
Derived
PS C:\Hoor

```
class Base
{
public:
    virtual std::string getName() const { return "Base"; }

    virtual ~Base() = default;
};
```

```
class Derived : public Base
{
public:
    std::string getName() const { return "Derived"; }

    virtual ~Derived() = default;
};
```



```
int main()
{
    Base* p { new Derived };

    std::cout << p->getName();

    delete p;
    p = nullptr;

    return 0;
}
```

As soon as the return type and signature get matched, **Derived::getName()** overrides **Base::getName()**. As such, writing both **virtual** and **override** are optional with **Derived::getName()**.

However, writing **override** is recommended. Helps to avoid mistakes and improves readability. Also, **override** implies **virtual**, so only writing **override** is sufficient.

Pure Virtual Functions

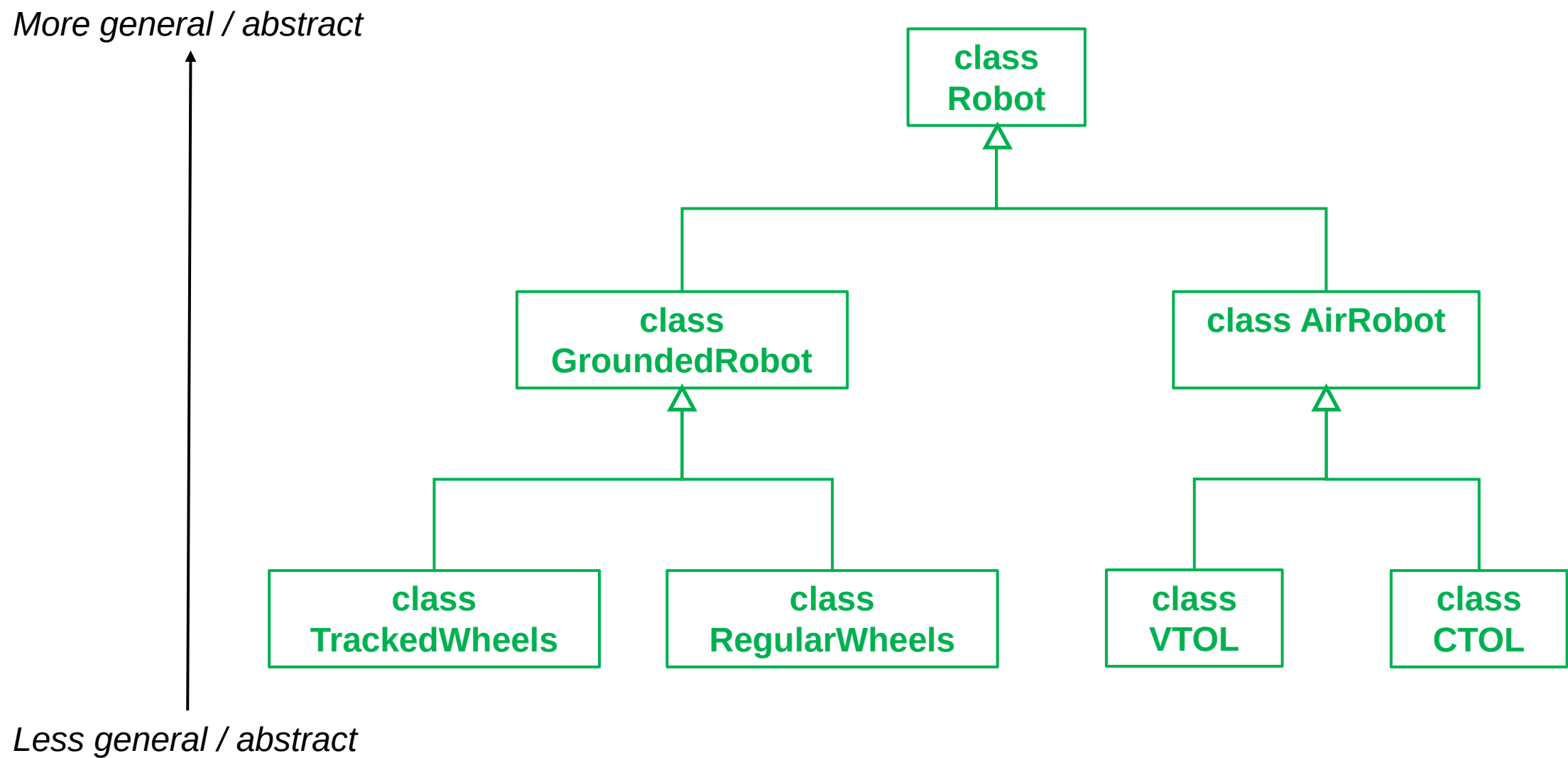
*Introduction to **Abstract Base Classes** and **Interfaces**.*

<https://www.learncpp.com/cpp-tutorial/pure-virtual-functions-abstract-base-classes-and-interface-classes/>

Abstract Base Classes

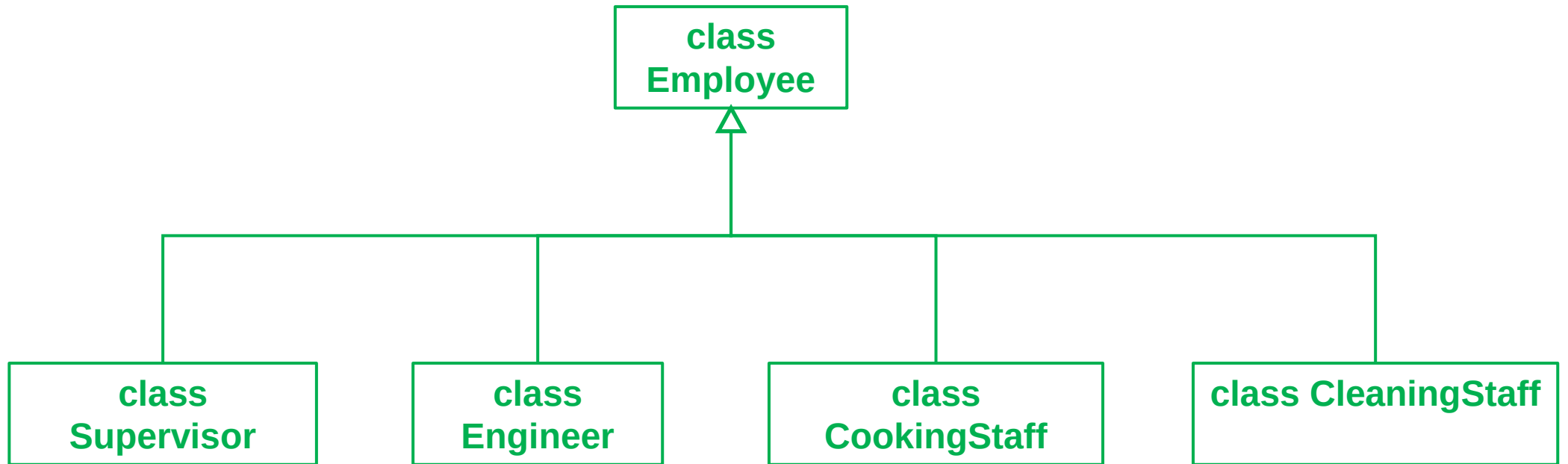
- Often times, when developing an inheritance hierarchy, you will find that instantiating objects of the *most-base* class won't make sense.

In this situation, you probably would never create an object of **class Robot**, because it is too general, and the objects that you'll want are probably more specific than this.



Similar situation here too.

More general / abstract



Less general / abstract

Abstract Base Classes (ABC)

- An ABC contains at least one **pure virtual function**.
 - One effect of this is that objects of ABC **cannot** be instantiated.
- What is a **pure virtual function**?
 - Such functions *(usually)* have no definitions (i.e. function bodies).
- What is the point of having a function without a definition?

Abstract Base Classes

- Consider a function **move(...)** inside the general **Robot** class that we saw earlier.
 - Can you comment on how should it be implemented?
- It's difficult, since robots can roll, walk, fly, swim, etc.
- A general **Robot** class cannot determine how **move** will be implemented.
 - Therefore, you can make these functions pure virtual; no need to provide an implementation.
- So then who provides the details for **move**?
 - When we add a pure virtual function to our class, we are effectively saying, *“it is up to the derived classes to implement this function”*.

These are what **pure virtual functions** look like, and how they compare with other types of member functions.

Note that a function body isn't required for **pure virtual methods**.

```
class Base
{
public:
    std::string sayHi() const { return "Hi"; } // a normal non-virtual function

    virtual std::string getName() const { return "Base"; } // a normal virtual function

    ➡ virtual int getValue() const = 0 ; // a pure virtual function

    int doSomething() = 0 ; // Compile error: can not set non-virtual functions to 0
};
```

Something is missing in this class. Can you spot it?

*If I have virtual methods, I intend on deriving more classes from **class Base**. Which means...*

*I should make **Base::~~Base()** virtual!*

Abstract Base Classes (ABC)

- There are two main consequences of introducing a pure virtual function.
 1. Any class with at least one pure virtual function becomes an **abstract base class**, which means that it **cannot be instantiated!**
 - You cannot create objects of an abstract base class.
 2. Any derived class should define a body for this function.
 - If not, then that derived class will also be considered an abstract base class.
 - So can't create objects.

You can no longer instantiate objects of type **Base**.

```
class Base
{
public:
    std::string sayHi() const { return "Hi"; }

    virtual std::string getName() const { return "Base"; }
```

Again, **Base::~~Base()**
should be virtual here.
By default, it isn't virtual.

→ virtual int getValue() const = 0;

```
    // int doSomething() = 0;
};
```

```
int main()
{
    Base b1;

    Base* p = new Base;
```

→ delete p;

```
    return 0;
}
```



```
extra -wconversion -Wsign-conversion -std=c++14 .\rough.cpp
.\rough.cpp:6:7: warning: 'class Base' has virtual functions and accessible non-virtual destructor [-Wnon-virtual-dtor]
    class Base
```

```
    ^~~~
```

```
.\rough.cpp: In function 'int main()':
```

```
.\rough.cpp:29:10: error: cannot declare variable 'b1' to be of abstract type 'Base'
    Base b1;
```

```
    ^~~
```

```
.\rough.cpp:6:7: note: because the following virtual functions are pure within 'Base':
    class Base
```

```
    ^~~~
```

```
.\rough.cpp:13:17: note: 'virtual int Base::getValue() const'
```

```
    virtual int getValue() const = 0; // a pure virtual function
```

```
.\rough.cpp:31:19: error: invalid new-expression of abstract class type 'Base'
    Base* p = new Base;
```

```
    ^~~~
```

```
.\rough.cpp:32:12: warning: deleting object of abstract class type 'Base' which has non-virtual destructor will cause undefined behavior [-Wdelete-non-virtual-dtor]
    delete p;
```

```
    ^
```



Because you didn't define the pure virtual function in **Derived**, it is also considered **abstract**. Hence, you can't instantiate objects of **Derived** as well.

```
class Base
{
public:
    std::string sayHi() const { return "Hi"; }
```

```
    virtual std::string getName() const { return "Base"; }
```

➡

```
    virtual int getValue() const = 0;
```

```
    virtual ~Base() = default;
};
```

```
class Derived : public Base
{
public:
```

```
    std::string getName() const override { return "Derived"; }
```

```
    virtual ~Derived() = default;
};
```

```
int main()
```

```
{
```

```
    Derived d1;
```

```
    Derived* p { new Derived };
```

```
    delete p;
```

```
    return 0;
```

```
}
```

Overriding Base::getName(), however, is not required to make the derived class not abstract.

An example to further motivate the advantage of having an Abstract Base Class.

Since you are forced to define pure virtual functions in derived classes, this provides an inherent benefit. Let's see what that looks like.

```
class Animal
{
protected:
    std::string m_name;

    // Prevents us from making Animal objects in public.
    // Derived classes can still use this.
    Animal(std::string name)
        : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???" ; }

    virtual ~Animal() = default;
};
```

```
class Cat: public Animal
{
public:
    Cat(std::string name)
        : Animal{ name }
    {}

    std::string speak() const override
    { return "Meow"; }
};
```

```
class Dog: public Animal
{
public:
    Dog(std::string name)
        : Animal{ name }
    {}

    std::string speak() const override
    { return "Woof"; }
};
```

- **Animal::~~Animal()** is virtual.
 - **~Cat()** and **~Dog()** become virtual as a consequence.
- **virtual std::string speak() const** is virtual, and is being overridden in **Cat** and **Dog**.
- **Animal::Animal()** is protected – can't make its objects in public.
 - **(SS) Could I have set it to private?**

```
class Animal
```

```
{
protected:
    std::string m_name;
```

```
// Prevents us from making Animal objects in public.
// Derived classes can still use this.
```

```
Animal(std::string name)
: m_name{ name }
{ }
```

```
public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???"; }

    virtual ~Animal() = default;
};
```

```
class Cat: public Animal
{
public:
    Cat(std::string name)
        : Animal{ name }
    { }
```

```
    std::string speak() const override
    { return "Meow"; }
};
```

```
class Dog: public Animal
{
public:
    Dog(std::string name)
        : Animal{ name }
    { }
```

```
    std::string speak() const override
    { return "Woof"; }
};
```


(SS) As an example, if **Animal::Animal()** is private, it can't be called from within **Cat::Cat()** and **Dog::Dog()**. So the derived objects cannot be constructed.

```
class Cat: public Animal
{
public:
    Cat(std::string name)
        : Animal{ name }
    {}

    std::string speak() const override
    { return "Meow"; }
};

int main()
{
    Cat cat1{ "tom" };

    return 0;
}
```

```
x86_64 -fno-conversion -fsigned-conversion -std=c++14 -I. \rough.cpp
```

```
.\rough.cpp: In constructor 'Cat::Cat(std::__cxx11::string)':
.\rough.cpp:30:24: error: 'Animal::Animal(std::__cxx11::string)' is private within
this context
```

```
        : Animal{ name }
          ^
```

```
.\rough.cpp:12:5: note: declared private here
```

```
    Animal(std::string name)
    ^~~~~~
```

```
.\rough.cpp: In constructor 'Dog::Dog(std::__cxx11::string)':
```

```
.\rough.cpp:41:24: error: 'Animal::Animal(std::__cxx11::string)' is private within
this context
```

```
        : Animal{ name }
          ^
```

```
.\rough.cpp:12:5: note: declared private here
```

```
    Animal(std::string name)
    ^~~~~~
```

```
D:\C++Users\mohamed_celmer\OneDrive - Ushib University\Documents\Source Stuff\Fc1
```

Now what if I derive a new class from **Animal**, and forget to redefine **speak()**?

Let's run through this program.

```
class Animal
{
protected:
    std::string m_name;

    Animal(std::string name)
    : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???" ; }

    virtual ~Animal() = default;
};
```

Protected, so accessible.

```
class Cow : public Animal
{
public:
    Cow(std::string name)
    : Animal{ name }
    {}

    // We forgot to redefine speak
};

int main()
{
    Animal* pAni{ new Cow{"Betsy"} };

    std::cout << pAni->getName() << " says "
               << pAni->speak() << '\n';

    return 0;
}
```

Now what if I derive a new class from **Animal**, and forget to redefine **speak()**?

Let's run through this program.

```
class Animal
{
protected:
    std::string m_name;

    Animal(std::string name)
        : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???" ; }

    virtual ~Animal() = default;
};
```

Not virtual

```
class Cow : public Animal
{
public:
    Cow(std::string name)
        : Animal{ name }
    {}

    // We forgot to redefine speak
};

int main()
{
    Animal* pAni{ new Cow{"Betsy"} };

    std::cout << pAni->getName() << " says "
              << pAni->speak() << '\n';

    return 0;
}
```

Now what if I derive a new class from **Animal**, and forget to redefine **speak()**?

Let's run through this program.

```
class Animal
{
protected:
    std::string m_name;

    Animal(std::string name)
        : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???" ; }

    virtual ~Animal() = default;
};
```

```
class Cow : public Animal
{
public:
    Cow(std::string name)
        : Animal{ name }
    {}

    // We forgot to redefine speak
};

int main()
{
    Animal* pAni{ new Cow{"Betsy"} };

    std::cout << pAni->getName() << " says "
               << pAni->speak() << '\n';

    return 0;
}
```

Now what if I derive a new class from **Animal**, and forget to redefine **speak()**?

Let's run through this program.

```
class Animal
{
protected:
    std::string m_name;

    Animal(std::string name)
        : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const
    { return "???" ; }

    virtual ~Animal() = default;
};
```

Betsy says ???

*This is the most-
derived
implementation*

```
class Cow : public Animal
{
public:
    Cow(std::string name)
        : Animal{ name }
    {}

    // We forgot to redefine speak
};

int main()
{
    Animal* pAni{ new Cow{"Betsy"} };

    std::cout << pAni->getName() << " says "
               << pAni->speak() << '\n';

    return 0;
}
```

If we make **Animal::speak()** pure virtual, we must define **Cow::speak()** so that its objects can be created.

- We're forced to define the function in **Cow** and cannot possibly forget.
- Keeping **Animal::Animal()** protected is no longer a requirement – we can't create **Animal** type objects because of pure virtual function.

```
class Animal
{
protected:
    std::string m_name;

    Animal(std::string name)
        : m_name{ name }
    {}

public:
    std::string getName() const
    { return m_name; }

    virtual std::string speak() const = 0;
    // { return "???"; }

    virtual ~Animal() = default;
};
```

Betsy says Moo

```
class Cow : public Animal
{
public:
    Cow(std::string name)
        : Animal{ name }
    {}

    std::string speak() const override
    { return "Moo"; }
};

int main()
{
    Animal* pAni{ new Cow{"Betsy"} };

    std::cout << pAni->getName() << " says "
              << pAni->speak() << '\n';

    return 0;
}
```

I hope you noticed an error in
the last discussion...

Hint: C++ may not complain about this error.

Interfaces

Interface Classes

- Interfaces don't exist in C++ in the way they exist in other languages.
- They are even more general than ABCs.

Interface Classes

- Have **no member variables** (i.e. attributes).
- **All member functions are pure virtual.**
 - Interfaces tend to be very general as a consequence.
- Class names are conventionally prefixed with an **'I'**.
 - **class IAnimal** (assuming this was an interface).
- They're easy to use and maintain.
- *An example on interfaces is discussed in the linked reading.*
 - <https://www.learncpp.com/cpp-tutorial/pure-virtual-functions-abstract-base-classes-and-interface-classes/>
 - *It is a brand new example.*

Interface Classes

- Interfaces will have virtual destructors just like we've seen before.
 - Note that even in an interface class, the destructor must have an implementation.
- Some languages like Java and C# have a keyword, **interface**, for making interface classes.
 - So you don't have to go and make every method pure virtual.
- Within these other languages, interfaces allow for multiple inheritance.
 - For multiple inheritance in C++, interfaces are not required.